

Practice 1: Create a Database and Collection

Step 1

Log in to MongoDB Atlas.

Step 2

Open your cluster.

Step 3

Click **Browse Collections**.

You will probably see:

No databases found

That's okay.

Step 4

Click **+ Create Database**

Fill in:

Database Name

college

Collection Name

student

Click **Create**.

Now you have:

college

|

└─ student

Practice 2: Insert Your First Document

Click the **student** collection.

Click **Insert Document**.

Paste:

```
{  
  "name": "Rahul",  
  "age": 20,  
  "city": "Chennai"  
}
```

Click **Insert**.

You should now see something like:

```
{  
  "_id": ObjectId(...),  
  "name": "Rahul",  
  "age": 20,  
  "city": "Chennai"  
}
```

Notice the `_id` field. MongoDB automatically creates it if you don't provide one.

Practice 3: Insert Another Document

Click **Insert** again and add:

```
{  
  "name": "Priya",  
  "age": 21,  
  "city": "Bangalore"  
}
```

Now your collection contains two documents:

Student Collection

Document 1

Rahul

20

Chennai

Document 2

Priya

21

Bangalore

Practice 4: Edit a Document

Find Rahul's document.

Click the **Edit** (pencil) icon.

Change:

"age": 20

to

"age": 22

Save the changes.

Practice 5: Delete a Document

Delete Priya's document using the **Delete** (trash) icon.

Now only Rahul's document should remain.

What You've Learned

By doing these exercises, you've already practiced the concepts behind:

- Creating a database
- Creating a collection
- Inserting a document
- Updating a document
- Deleting a document

These are the same CRUD operations you'll later perform with mongosh using commands like `insertOne()`, `find()`, `updateOne()`, and `deleteOne()`.

CRUD Operations (Create, Read, Update, Delete)

CRUD is asked in almost every MongoDB interview.

What is CRUD?

CRUD stands for:

Letter	Meaning	MongoDB Command
C	Create	<code>insertOne()</code> , <code>insertMany()</code>
R	Read	<code>find()</code> , <code>findOne()</code>
U	Update	<code>updateOne()</code> , <code>updateMany()</code>
D	Delete	<code>deleteOne()</code> , <code>deleteMany()</code>

Think of it like a student database:

Student Collection

Rahul

Priya

Arun

You can:

- Add a student (Create)
- View students (Read)
- Change a student's details (Update)
- Remove a student (Delete)

CREATE - insertOne()

Syntax

```
db.collection.insertOne(  
  {  
    field1: value1,  
    field2: value2  
  }  
)
```

Here:

- db → current database
- student → collection
- insertOne() → inserts one document

Example

```
db.student.insertOne({
  name: "Rahul",
  age: 20,
  city: "Chennai"
})
```

What is _id?

Every MongoDB document has a unique _id.

Example:

```
{
  "_id": ObjectId("686bcd123456789"),
  "name": "Rahul"
}
```

It uniquely identifies the document.

Just like a **Primary Key** in SQL.

Output of insertOne()

```
{
  acknowledged: true,
  insertedId: ObjectId("686bcd...")
}
```

Meaning:

- acknowledged: true → Insert was successful.
- insertedId → ID of the inserted document.

CREATE - insertMany()

Used to insert multiple documents at once.

Syntax

```
db.student.insertMany([  
  { ... },  
  { ... },  
  { ... }  
])
```

Notice the square brackets []. They represent an **array** of documents.

Example

```
db.student.insertMany([  
  {  
    name: "Rahul",  
    age: 20,  
    city: "Chennai"  
  },  
  {  
    name: "Priya",  
    age: 21,  
    city: "Bangalore"  
  },  
  {  
    name: "Arun",  
    age: 19,  
    city: "Hyderabad"
```

```
}  
])
```

After Inserting

Student Collection

Rahul

Priya

Arun

Output

```
{  
  acknowledged: true,  
  insertedIds: {  
    '0': ObjectId(...),  
    '1': ObjectId(...),  
    '2': ObjectId(...)  
  }  
}
```

Each inserted document gets its own `_id`.

Difference: `insertOne()` vs `insertMany()`

`insertOne()`

Inserts one document

Uses {}

`insertMany()`

Inserts multiple documents

Uses [{},{},{}]

READ - find()

Used to display documents.

Syntax

```
db.student.find()
```

Example

Suppose your collection contains:

```
{  
  "name": "Rahul",  
  "age": 20  
}  
  
{  
  "name": "Priya",  
  "age": 21  
}
```

Run:

```
db.student.find()
```

Output:

Rahul

Priya

(Actual output will include `_id` and other fields.)

READ - findOne()

Returns only the **first matching document**.

```
db.student.findOne()
```

Output:

```
{  
  "name": "Rahul",  
  "age": 20  
}
```

Only one document is returned.

Difference: find() vs findOne()

find()

Returns all matching documents

Output is a cursor

findOne()

Returns the first matching document

Output is a single document

Placement Interview Questions

Q1. What is CRUD?

Answer:

CRUD stands for Create, Read, Update, and Delete. These are the four basic operations used to manage data in MongoDB.

Q2. Difference between insertOne() and insertMany()?

Answer:

- insertOne() inserts one document.
 - insertMany() inserts multiple documents in a single operation.
-

Q3. What is _id?

Answer:

_id is a unique identifier automatically added to every MongoDB document. It is similar to the primary key in SQL.

Q4. Difference between find() and findOne()?

Answer:

- find() returns all matching documents.
- findOne() returns only the first matching document.

MongoDB Placement Preparation – Part 4

find() with Filters (Query Documents)

In the previous lesson, you learned:

```
db.student.find()
```

This returns **all documents**.

Now we'll learn how to retrieve **specific documents**.

1. Find All Documents

```
db.student.find()
```

2. Find by Equality

Find students whose age is 20

```
db.student.find({ age: 20 })
```

MongoDB checks every document:

Name Age Match?

Rahul 20 

Priya 21 

Name Age Match?

Arun 19 ✖

Kavya 22 ✖

Vijay 20 ✔

Output:

Rahul

Vijay

Find students from Chennai

```
db.student.find({ city: "Chennai" })
```

Comparison Operators

These are very commonly asked.

Operator	Meaning
\$gt	Greater Than
\$gte	Greater Than or Equal
\$lt	Less Than
\$lte	Less Than or Equal
\$ne	Not Equal
\$in	Value is in a list

3. Greater Than (\$gt)

Find students older than 20.

```
db.student.find({  
  age: { $gt: 20 }  
})
```

4. Greater Than or Equal (\$gte)

Find students with age 20 or more.

```
db.student.find({  
  age: { $gte: 20 }  
})
```

5. Less Than (\$lt)

Find students younger than 20.

```
db.student.find({  
  age: { $lt: 20 }  
})
```

6. Less Than or Equal (\$lte)

```
db.student.find({  
  age: { $lte: 20 }  
})
```

7. Not Equal (\$ne)

Find everyone except students aged 20.

```
db.student.find({  
  age: { $ne: 20 }  
})
```

```
}}
```

8. \$in Operator

Suppose you want students from either Chennai or Mumbai.

Instead of writing two queries, use \$in.

```
db.student.find({  
  city: {  
    $in: ["Chennai", "Mumbai"]  
  }  
})
```